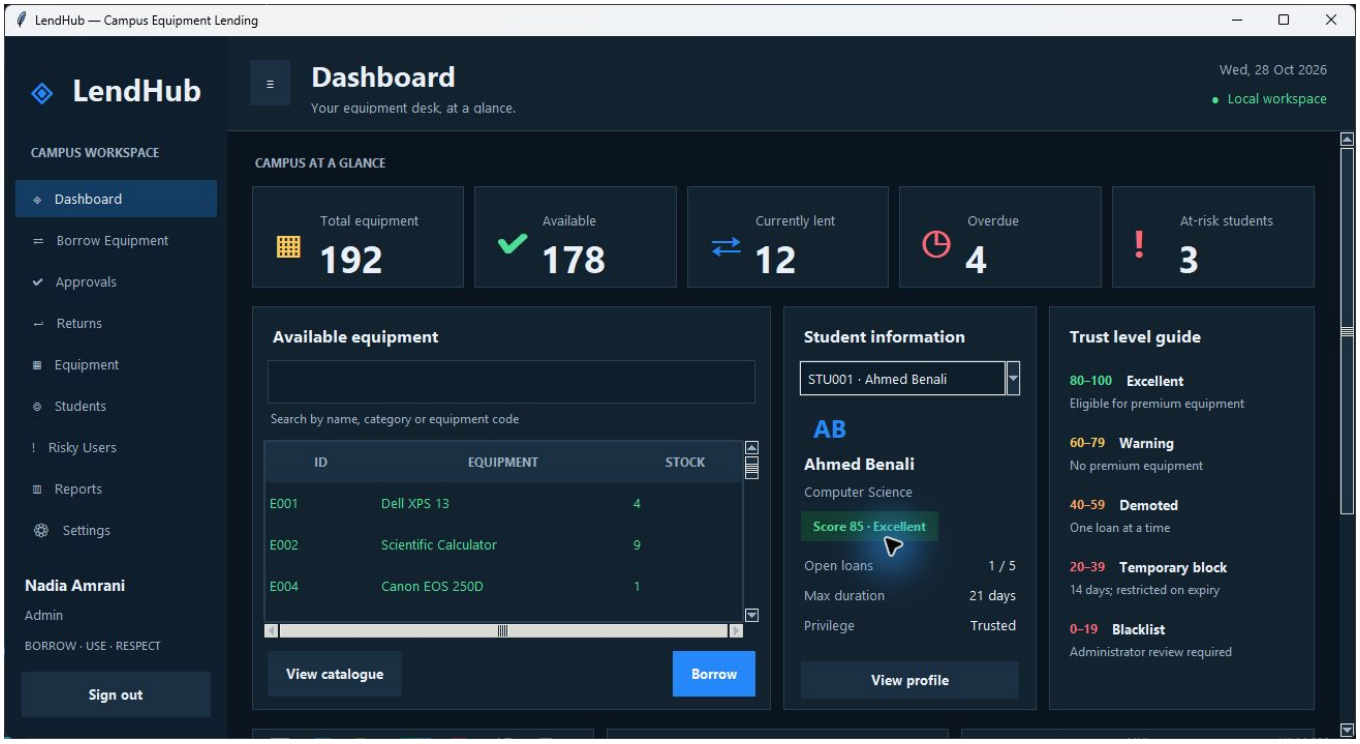
Shared tools. Trusted access.

LendHub

A desktop application for fair, accountable student access to campus equipment.

The application connects a searchable equipment catalogue with request approvals, returns and a transparent trust score. Students understand their allowance; staff can see stock and sanctions before committing a loan.



Administrator dashboard, captured from the running Windows application. Counts are calculated from the database.

Accounts	Equipment types	Physical units	Demo loans
20	40	192	60

Delivery: runnable Python project, shared rules, three-table schema, automated tests, screenshots, sample exports and launch instructions. The project runs offline with Python 3.10+ and Tkinter; no pip packages are required.

Database design

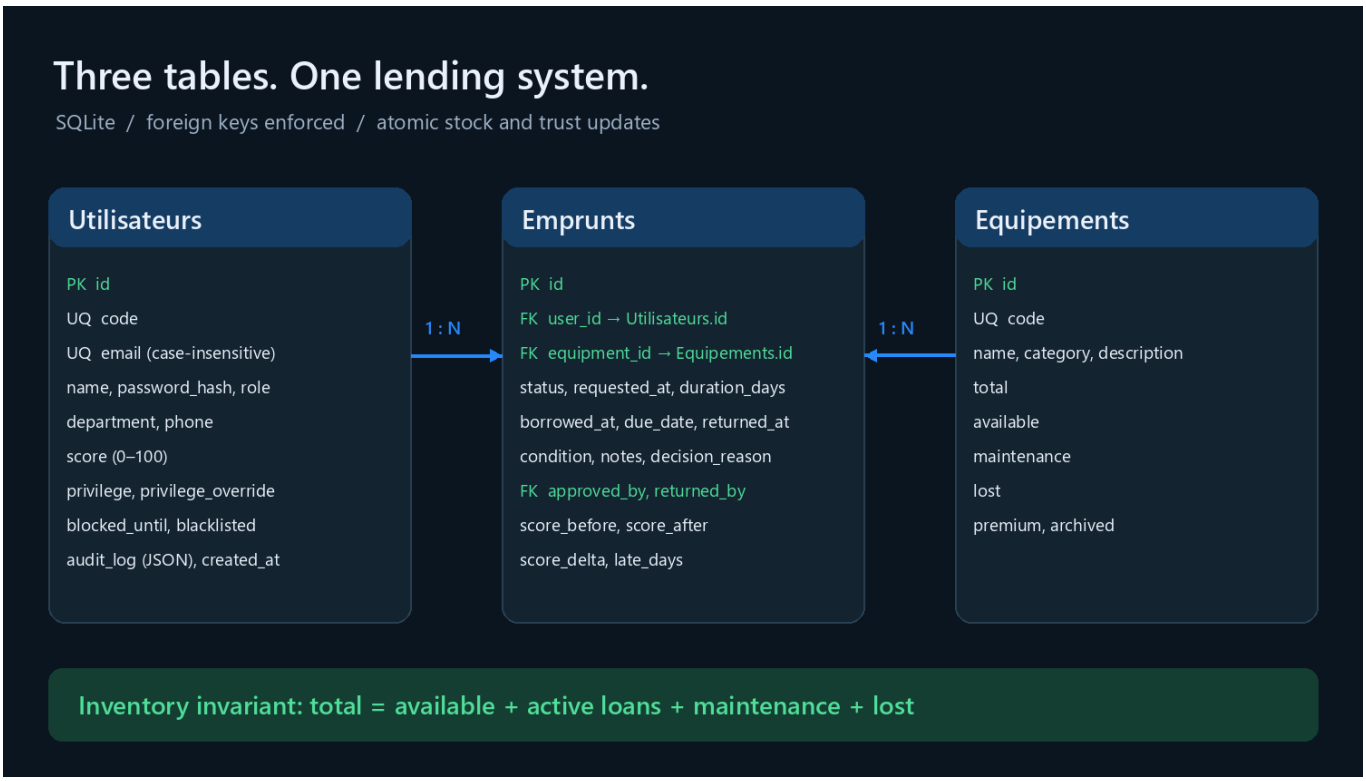


Table	Purpose
Utilisateurs	Students and staff; identity, salted password hash, trust score, sanctions and review history.
Equipements	Equipment types and quantities: total, available, maintenance and lost units.
Emprunts	Request/approval/return lifecycle, due dates, condition and score snapshots.

Integrity and consistency

Foreign keys link each loan to one student and one equipment type. Optional staff foreign keys record approval and return handling. Unique email and code constraints prevent duplicates. CHECK constraints protect role/status values, score ranges, dates and nonnegative quantities.

Stock-changing actions run inside **BEGIN IMMEDIATE** transactions. Approval rechecks current stock and eligibility before decrementing availability. Return processing updates equipment, loan history and trust score together, or rolls all changes back.

The inventory invariant is **total = available + active loans + maintenance + lost**. User review history is stored in a JSON field to retain the required three-table design.

Lending and trust policy

Privilege	Open + pending	Max duration	Premium
Restricted	1	7 days	No
Standard	3	14 days	No
Trusted	5	21 days	Yes

Score	Level	Borrowing effect
80-100	Excellent	Trusted privileges
60-79	Warning	Standard; no premium equipment
40-59	Demoted	Restricted to one loan
20-39	Temporary block	14 days after a negative return; restricted on expiry
0-19	Blacklist	Administrator review required

Return event	Score change
Good and on time	+2
Late	-5 per day, capped at -60
Damaged	-25 plus any late penalty
Lost	-60 plus any late penalty

Scores are clamped to 0-100. New accounts start at 85. Pending requests consume allowance but do not reserve stock. Every loan requires staff approval. Overdue equipment prevents further borrowing.

Temporary blocks expire on their expiry date. An administrator review requires a reason; unblocking clears sanctions and raises scores below 40 to 40. Privilege overrides are caps and cannot bypass trust restrictions. See docs/rules.md for full boundary behavior.

Architecture and implementation

Layer	Responsibility
main.py + config/	Startup, local paths and centralized rules
ui/	Tkinter pages, navigation, reusable cards/tables/dialogs
services/	Authentication, eligibility, trust, inventory, reports and exports
database/repositories/	Parameterized SQL and row access
database/connection.py	SQLite connections, foreign keys and transaction boundaries

Approval transaction

The following excerpt is taken from the delivered loan service. It illustrates validation immediately before stock reservation.

```
def approve(self, loan_id):
    actor = self.auth.require('technician','admin')
    with self.db.transaction() as conn:
        loan = loans.get(conn,loan_id)
        if not loan or loan['status'] != 'pending': raise ValidationError('Only pending requests c
        self._eligible(conn,loan['user_id'],loan['equipment_id'],loan['duration_days'],exclude=loa
        if not equipment.reserve(conn,loan['equipment_id']): raise ValidationError('The last avail
        today = self.today()
        loans.update(conn,loan_id,status='active',borrowed_at=today.isoformat(),due_date=(today+ti
```

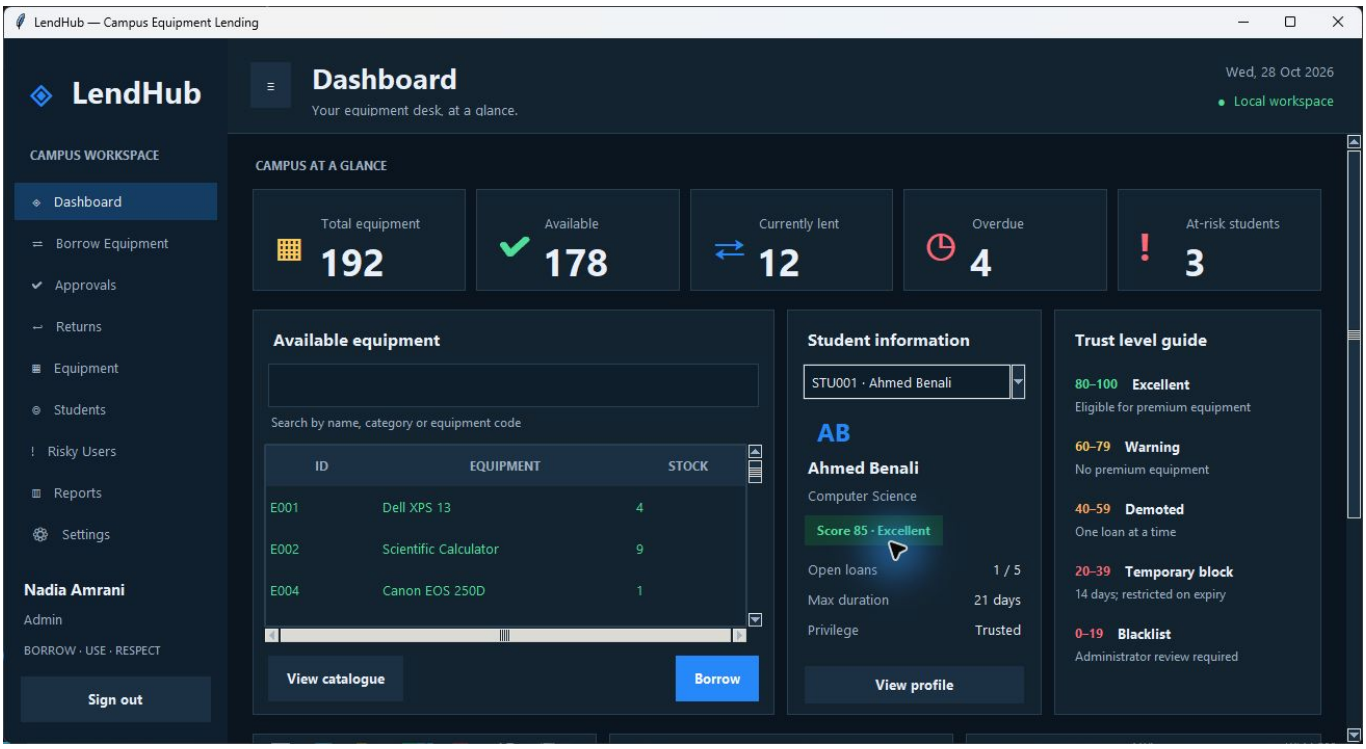
Authentication and exports

Passwords use PBKDF2-HMAC-SHA256 with 260,000 iterations, random 16-byte salts and constant-time digest comparison. Service methods enforce roles; students can only access their own profile and loans.

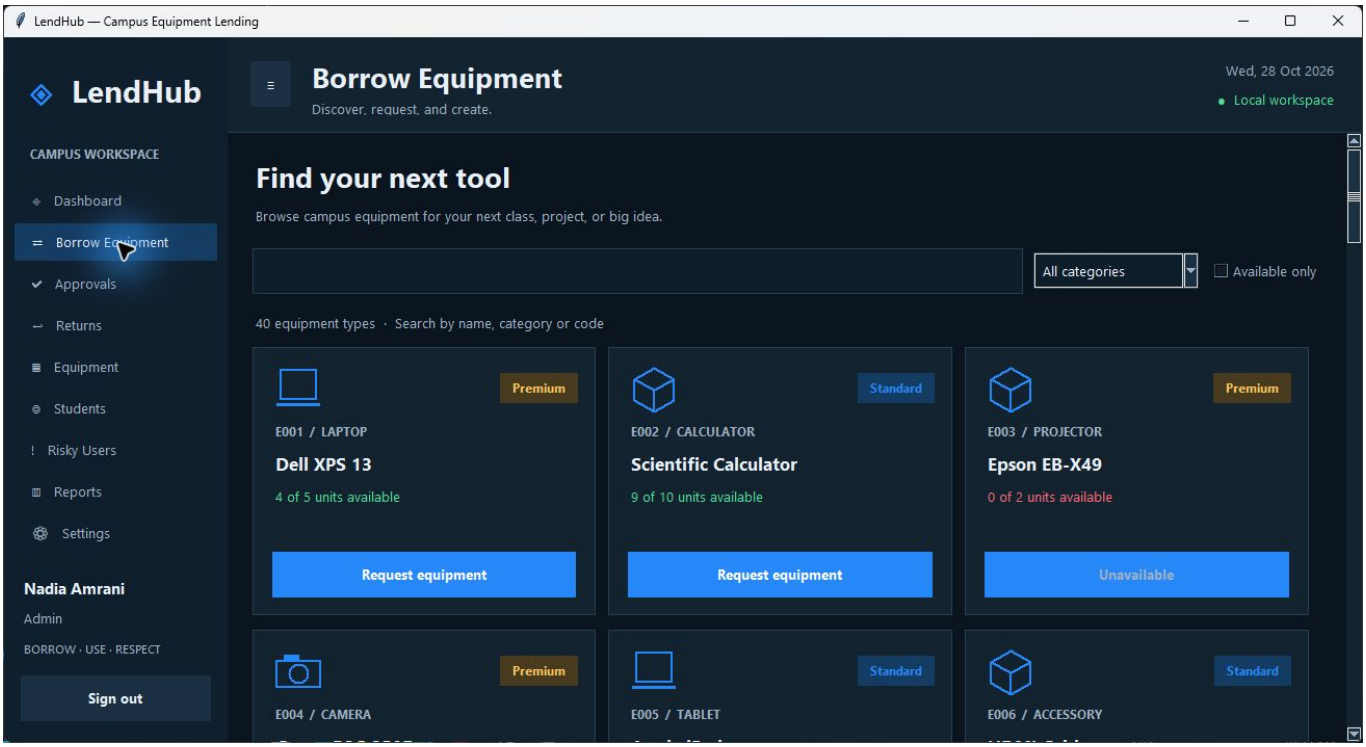
PDF slips use the PDF standard directly. QR labels use a small version 2-L byte-mode encoder and standard-library PNG writing. CSV import is transactional; report export escapes spreadsheet formula-like text. Slow imports and exports run in a background thread, with UI updates returned through a queue.

The UI does not query SQLite directly. A separate integration test suite verifies services without importing Tkinter.

Interfaces: dashboard and catalogue

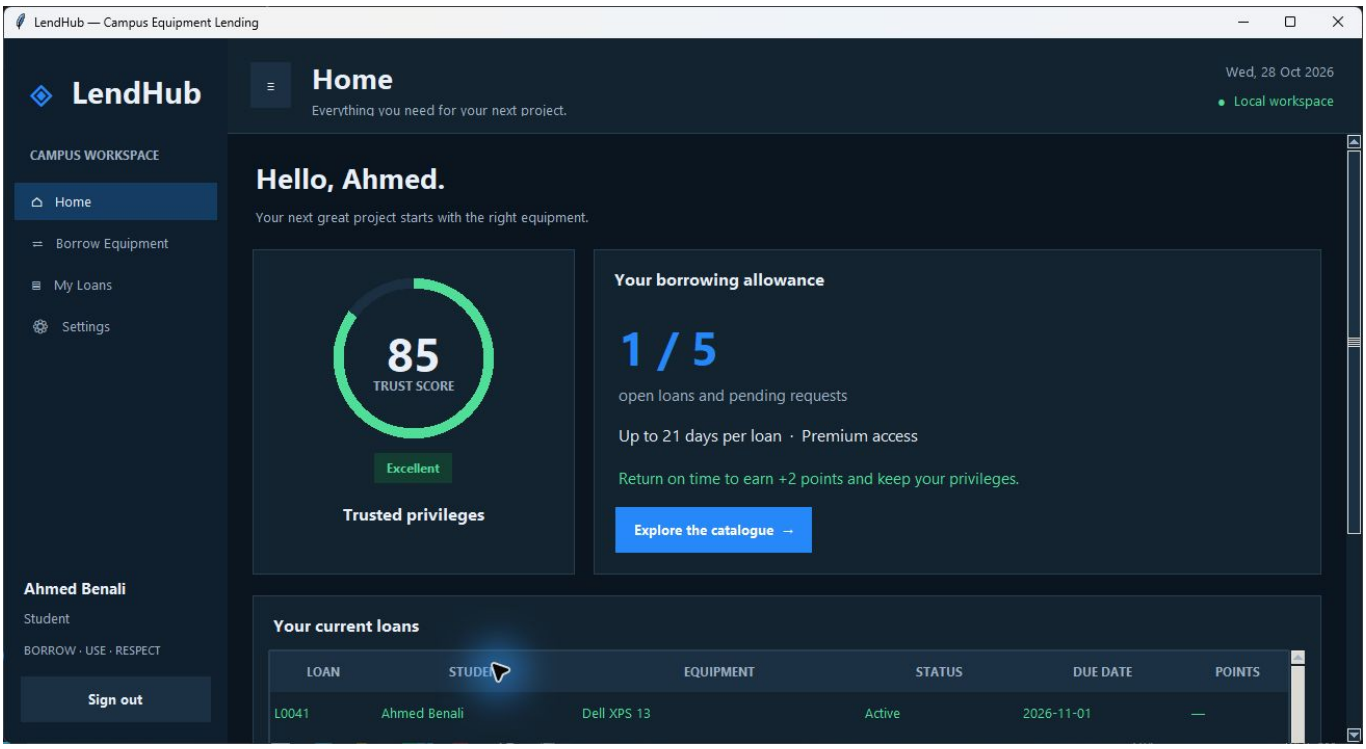


Dashboard. Live totals, equipment search, student allowance and the complete trust guide. The scrollable lower section contains monthly borrowing, top items and recent activity.

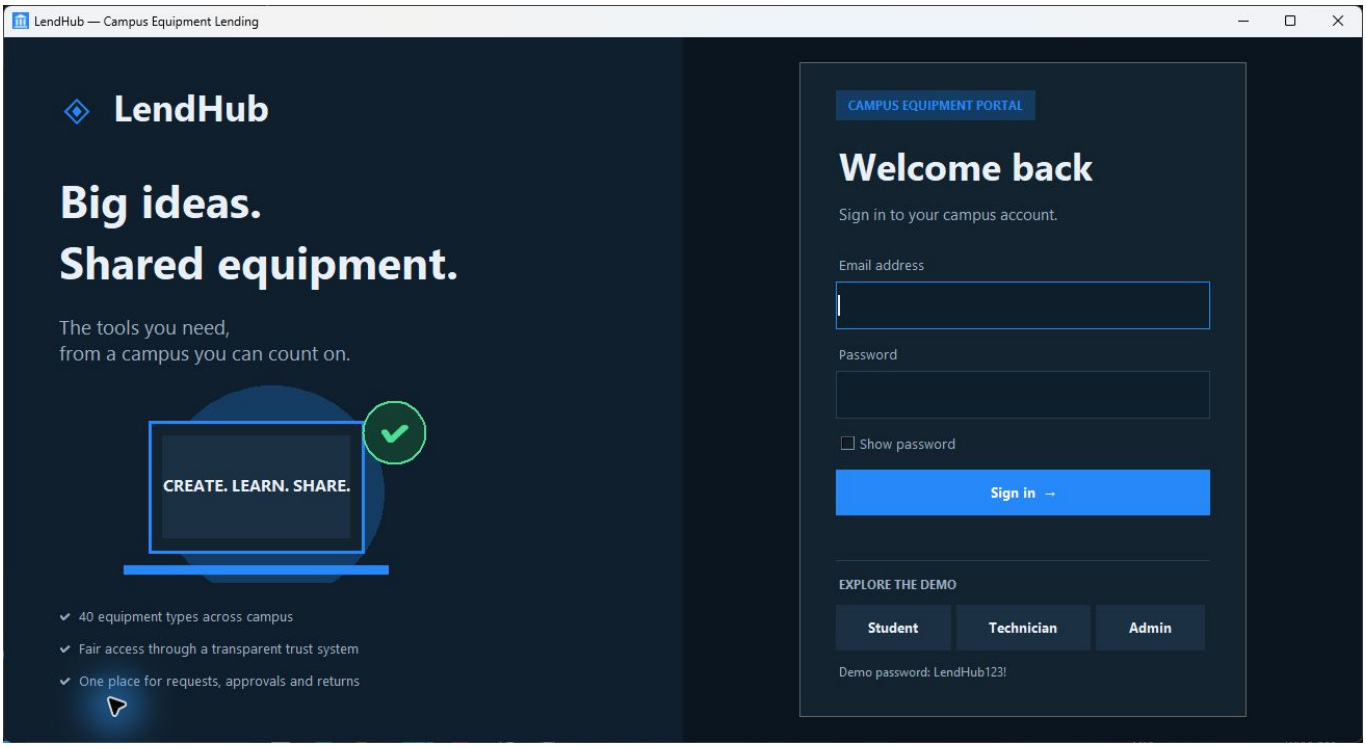


Catalogue. Equipment cards show stock, category and access tier. Search and filters update immediately. Unavailable or restricted requests cannot be submitted.

Interfaces: student and sign-in

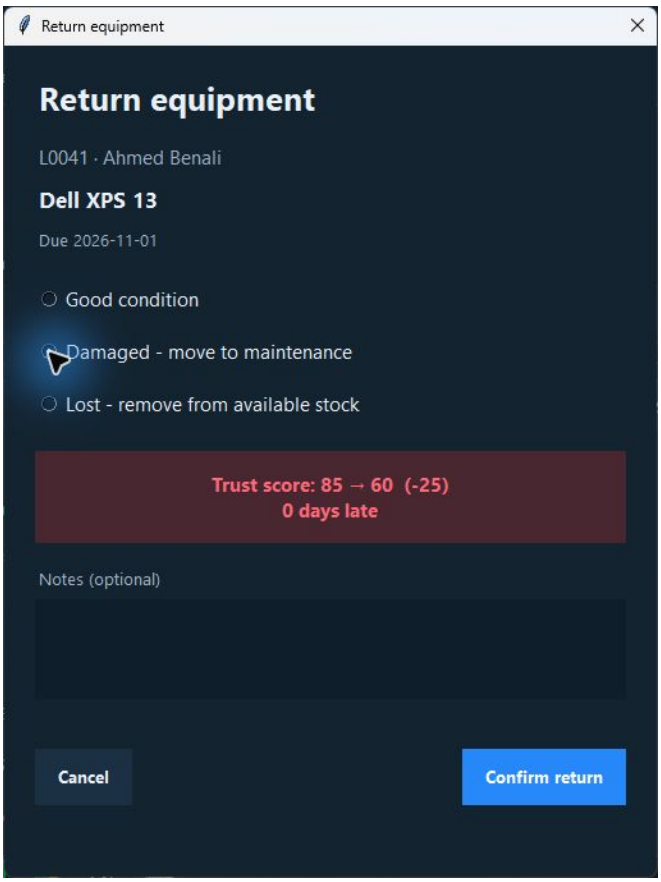


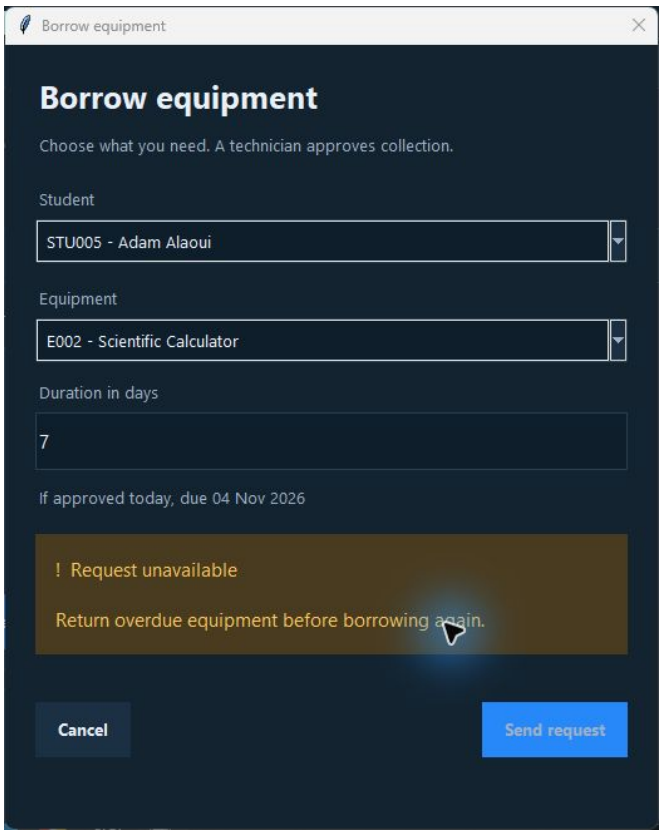
Student home. The trust gauge, allowance, due-date guidance and current loans make borrowing conditions visible before a request.



Sign-in. Clear credentials form, show/hide password and labeled fictional demo accounts. The sidebar changes by role after authentication.

Interfaces: decisions before commitment





Return preview (left). A damaged, on-time return changes Ahmed's score from 85 to 60. The unit goes to maintenance, not available stock. This screenshot previews the decision without committing a return.

Borrowing warning (right). A student with overdue equipment cannot place another request. The interface explains why and disables submission; the service checks the same rule independently.

Staff workflow

1. Review a pending request and inspect the student profile.
2. Approve collection or refuse with a reason.
3. At return, find the loan or scan the equipment QR value.
4. Select condition, review the score change and confirm.
5. Administrators can review sanctions and record a reason for any override.

Confirmation dialogs protect decisions. Request rejection, cancellation and duplicate-return checks prevent invalid lifecycle transitions. The return desk is available to both technicians and administrators.

Verification and delivery

Check	Result
Service and integration tests	40 tests passed
Role/theme GUI smoke test	56 page and dialog checks passed
Concurrency	Only one approval can take the last unit
QR interoperability	Generated label independently decoded with OpenCV
PDF export	Parsed successfully and visually reviewed after rendering
Native Windows review	Dashboard, catalogue, student, technician, login and return dialogs inspected

Run and demonstrate

Extract the full project folder. On Windows double-click **Start LendHub.cmd**, or run **python main.py**. Use admin@lendhub.local, tech@lendhub.local or student@lendhub.local with password **LendHub123!**. Demo records are fictional.

Run **python -m unittest discover -s tests -v** for the automated suite, and **python tests/ui_smoke.py** for interface checks. The smoke test uses a separate temporary database. Reset only a disposable demo using **python -m database.seed --reset --confirm-reset**.

Limitations and next steps

This is a local academic application. The computer owner can access the SQLite file; app permissions are not an operating-system security boundary. There is no cloud sync, email service or camera scanner. PDF text uses a built-in Latin font. Dates use the local system clock.

Possible extensions include a dedicated audit-event table, account provisioning, a camera scanner, multilingual PDF fonts, server-backed sessions and automated database migrations. These are outside the delivered offline scope.

Academic handover

The source, schema, rules, screenshots and sample exports are included. Add your actual institution, team names, repository link and contribution evidence before an academic submission. No team history or submission status has been fabricated.

Basis: the supplied LendHub roadmap and interface reference, with implementation choices documented in README.md and docs/rules.md.